Rust vs. C++ for CPU and GPU Performance: A Controlled Empirical Benchmark Study*

Aditya Raut[1], Shelley Wong[2]
[1]Computer Information Systems
[2]Computer Science Department
California State University, Chico
Chico, CA 95929
{araut1,swong26}@csuchico.edu

Abstract

This paper presents a controlled empirical study comparing the runtime performance of Rust and C++ across CPU computational benchmarks and CUDA-accelerated GPU workloads. Benchmarks cover sorting, search, hash tables, dense matrix multiplication, fast Fourier transform (FFT), n-body simulation, and reduction kernels. All experiments were executed on a dedicated workstation equipped with an Intel Core Ultra 7 processor and an NVIDIA RTX 5060 Ti GPU running Ubuntu 24.04 and CUDA 12.4. Statistical analysis applies Welch's t-test, Cohen's d effect size, and 95 % confidence intervals to 5–30 timed trials per benchmark. Rust outperforms C++ on several CPU tasks—notably search, sorting, and integer hash tables—while C++ retains advantages on FFT, naive matrix multiplication, and one parallel-reduction configuration. For the GPU kernels, Rust achieves significantly lower kernel times for matrix multiplication ($p = 0.017$, $d = -2.49$) and reduction ($p = 0.001$, $d = -5.51$), while the softmax comparison yields no statistically significant difference ($p = 0.778$). Safety-oriented metadata confirm zero `unsafe` blocks in the measured Rust paths, and Rust requires fewer lines of code in aggregate. The results are consistent with the hypothesis that Rust can be performance-competitive across both CPU and GPU workloads without sacrificing safety guarantees.

*CSCI 693 Research Methods in Computer Science, Spring 2026

1 Introduction

1.1 Context and Background

High-performance computing (HPC) continues to rely heavily on low-level programming languages that afford explicit control over memory layout, vectorisation, and hardware access [2]. C++ remains the dominant implementation language in many performance-critical CPU and GPU systems because of its mature compiler ecosystem (Clang/GCC/MSVC), direct CUDA integration, and an extensive body of optimised libraries [10]. Rust, however, has emerged as a strong alternative for systems programming by combining ahead-of-time compilation with a memory-safe ownership model and first-class concurrency guarantees [6].

1.2 Problem Statement and Research Questions

The core question for practitioners is whether Rust can offer comparable runtime performance to C++ while reducing safety risks and improving maintainability. This study operationalises that question through three hypotheses:

1. **H1 (CPU parity):** Rust runtimes on CPU workloads are within ±15 % of equivalent C++ runtimes on average.
2. **H2 (GPU parity):** Rust CUDA-backed GPU kernel times are within 5 % of equivalent C++ CUDA kernel times on average.
3. **H3 (safety cost):** Competitive Rust performance is achieved without the use of `unsafe` blocks, indicating that safety guarantees carry no explicit opt-out cost in this benchmark set.

1.3 Significance of the Study

Prior work suggests that Rust can approach C++ performance on CPU workloads and may be viable for GPU-adjacent development when paired with CUDA back-ends or driver bindings [1, 5, 8]. However, no prior study has evaluated Rust and C++ together on both CPU and GPU tasks using a single, unified benchmark project with end-to-end statistical rigour and a consistent hardware environment [4]. This study fills that gap by providing a controlled, reproducible comparison tied to publicly committed artifacts.

1.4 Artifact-Based Methodology and Scope

All benchmarks in this study were executed on a single, dedicated workstation (hardware specifications in Section 3) and the raw timing outputs were committed to a public repository alongside the source code, analysis scripts, and

derived summary tables.[1] This paper reports findings drawn from those committed artifacts rather than rerunning benchmarks at manuscript-preparation time. This choice is deliberate: it ties every numerical claim to a specific, documented hardware configuration and software stack, and ensures bit-for-bit reproducibility with the archived data. Repeating the benchmarks on different hardware would introduce platform-specific variability and make it harder to compare results against the committed dataset. Section 3 documents all conditions under which the original runs were performed, and the repository provides the raw data and scripts needed to reproduce the analysis on the same or equivalent hardware.

2 Literature Review

2.1 Overview of Existing Research

Research on Rust's performance relative to C++ has grown substantially since Rust's 1.0 release in 2015. Early studies focused on validating whether the ownership model and borrow-checker impose runtime costs. Zhang et al. characterise Rust's runtime behaviour across a broad set of microbenchmarks and report that the compiler generates competitive code on most workloads [12]. Ivanov compares Rust with C++ on common numerical and data-structure routines and finds that Rust can match or exceed C++ performance on many optimised CPU tasks, with occasional overheads on workloads that rely heavily on compiler-specific auto-vectorisation [5].

The conversation has since extended to HPC-scale benchmarks. Martins et al. adapt the NAS Parallel Benchmarks to Rust and show that the performance gap between Rust and established HPC toolchains (primarily C and Fortran) is continuing to narrow, though some collective-communication workloads retain non-trivial overheads [8]. The Stack Overflow Developer Survey consistently ranks Rust as the most-admired language while noting that its adoption in production HPC contexts remains limited relative to C++ [9], which motivates studies like this one that quantify the actual performance cost of that adoption.

On the GPU programming side, Bychkov and Nikolskiy examine Rust as a language for GPU workloads and emphasise that fair comparisons require equivalent kernel implementations and consistent memory-transfer accounting [1]. Köpcke et al. introduce Descend, a safe GPU systems language inspired in part by Rust's type system, reflecting broader research interest in bringing safety guarantees to GPU programming [7]. Veytsman et al. provide an applied

[1] `https://github.com/adityaanilraut/Rust-vs-Cpp-for-CPU-and-GPU-Performance`

case study in computational physics, arguing that Rust can eliminate classes of memory-related defects while preserving execution performance [11].

2.2 Critical Evaluation

Taken together, the existing literature establishes three points of consensus: (1) Rust produces competitive CPU binaries on most workloads, (2) the borrow-checker and ownership model incur negligible runtime overhead for well-written code, and (3) GPU-oriented Rust work is nascent but promising. However, most studies evaluate either CPU or GPU performance—not both together in the same controlled environment—and few apply full statistical rigour (confidence intervals, effect sizes, and variance checks) to their timing comparisons. Bychkov and Nikolskiy's caution about equivalent kernels is methodologically important but rarely operationalised in published comparisons. The study by Fulton et al. further demonstrates that adopting Rust carries real developer-experience benefits alongside safety gains, but these are measured qualitatively rather than empirically [3].

2.3 Research Gap and Rationale

The gap this study addresses is the absence of a unified, hardware-specific, statistically rigorous benchmark across both CPU and GPU workloads using identical source repositories and analysis pipelines for Rust and C++. Existing work either (a) focuses only on CPU, (b) uses informal timing without confidence intervals or effect sizes, or (c) evaluates GPU performance in isolation from a broader CPU context. This study contributes a controlled comparison that covers both domains, applies Welch's t-tests and Cohen's d to quantify statistical significance and practical effect size, and also measures safety and code-complexity proxies alongside performance.

3 Methodology

3.1 Research Design

This is a controlled empirical benchmark study. Rust and C++ implementations of the same algorithms were compiled at the highest optimisation level on the same machine, run against identical input datasets, and timed over multiple independent trials. Statistical tests were then applied to the resulting distributions to evaluate each hypothesis.

3.2 Hardware and Software Environment

- **CPU:** Intel Core Ultra 7 (Meteor Lake, efficiency-core + performance-core hybrid)
- **GPU:** NVIDIA GeForce RTX 5060 Ti (Blackwell architecture)
- **Operating System:** Ubuntu 24.04 LTS (64-bit)
- **CUDA:** Version 12.4, `nvcc` V12.4.131 (built Thu Mar 28 02:18:24 PDT 2024)
- **C++ compiler:** GCC via CMake Release build (`-O2 -DNDEBUG`); CUDA kernels compiled with `nvcc -O2`
- **Rust compiler:** `rustc` stable channel, `cargo build --release` (`opt-level = 3`)
- **CPU benchmark harness:** Google Benchmark (C++) and the standard-library timing approach (Rust); inputs defined by `generate_datasets.py`

All C++ and Rust source files, build scripts, dataset generation utilities, and analysis scripts are included in the public repository.

3.3 Benchmark Configuration and Fairness

CPU benchmarks. Each CPU benchmark was executed under Google Benchmark's auto-iteration mode, which determines the number of repetitions needed to achieve a stable mean. The timing metric is wall-clock time per operation, not CPU time, to capture scheduler effects consistently across both implementations.

GPU benchmarks. Each GPU kernel was run for five independent trials using separately launched CUDA contexts. Timing covers kernel execution only; host-to-device (H2D) and device-to-host (D2H) transfer times are recorded separately and excluded from the primary kernel-time comparison (though they are available in the raw CSVs). This separation follows the guidance of Bychkov and Nikolskiy [1] that transfer costs should not be conflated with kernel throughput in language comparisons.

Fairness measures. Both language implementations use algorithmically equivalent code for each benchmark. The same binary input datasets, generated by a single shared Python script, are used for all C++ and Rust runs. Compiler optimisation flags are set to the highest standard release level for each ecosystem (`-O2`/`-O3` for C++, `opt-level = 3` for Rust). No language-specific SIMD intrinsics or assembly stubs were introduced.

3.4 Statistical Analysis

For each (benchmark, input-size) pair, the following statistics are computed by `scripts/analyze_results.py`:

- Mean and standard deviation across all timed trials
- 95 % confidence interval via the Student t-distribution
- Welch's two-sample t-test (does not assume equal variance)
- Cohen's d effect size (pooled standard deviation formula)
- Coefficient of variation (CV %) with a high-variance flag at CV > 5 %

Percentage overhead is defined as $(\mu_{\text{Rust}} - \mu_{\text{C++}})/\mu_{\text{C++}} \times 100$, so negative values indicate Rust is faster.

3.5 Lines-of-Code and Safety Metrics

Lines of code (LOC) are counted by `scripts/safety_analysis.py` using the following rules: a line is counted if and only if it is non-blank and does not begin with a single-line comment character (`//` for C++ and Rust) or a block-comment delimiter (`/*` or `*`). The `build/` and `target/` directories are excluded. The tool is applied to `.cpp`, `.h`, and `.hpp` files for the C++ components and to `.rs` files for the Rust components.

Rust `unsafe` blocks are detected by scanning every `.rs` file (excluding `target/`) for lines matching the regular expression `\bunsafe\b`. A "recorded unsafe block" therefore means a file-level occurrence of the keyword `unsafe` in the measured source paths; it does not include transitive `unsafe` inside dependencies. The phrase "measured paths" refers specifically to the source files under `cpu/rust/` and `gpu/rust/` in the repository.

3.6 Limitations

Several limitations affect how broadly these results can be generalised. First, the GPU benchmark set contains only five trials per kernel, which is smaller than the 30+ trials typically used for CPU measurements; this produces wider confidence intervals and higher CV % values for the GPU results. Second, the benchmark suite covers a focused selection of kernels (sorting, search, linear algebra, FFT, reduction) and does not represent all HPC or AI workloads. Third, all results are tied to a single hardware platform; results may differ on CPUs with different microarchitectures or GPUs of different generations. Fourth, LOC and `unsafe` counts are proxies for safety and maintainability—they are necessary but not sufficient evidence for claims about developer experience.

4 Results

4.1 CPU Benchmark Outcomes

Table 1 presents the complete CPU results drawn from the committed summary table. The table covers all 15 CPU benchmarks with mean runtimes,

percentage overhead, p-values (Welch's t-test), and Cohen's d. Negative overhead indicates that Rust is faster than C++; positive overhead indicates that C++ is faster.

Table 1: CPU performance results: C++ vs. Rust (all times in ms, mean ± std).

Benchmark	Size	C++ (ms)	Rust (ms)	Overhead
binary_search	1 000 000	7.20 ± 0.03	2.28 ± 0.00	−68.3 %
lower_bound	1 000 000	7.26 ± 0.01	2.10 ± 0.01	−71.1 %
hashmap_int_insert	100 000	3.78 ± 0.00	1.08 ± 0.00	−71.3 %
hashmap_int_lookup	100 000	1.20 ± 0.00	0.97 ± 0.00	−19.8 %
hashmap_string_insert	100 000	8.39 ± 0.07	6.16 ± 0.02	−26.6 %
quicksort_std_sort	1 000 000	49.70 ± 0.04	9.62 ± 0.02	−80.6 %
quicksort_manual	1 000 000	54.47 ± 0.13	44.84 ± 0.04	−17.7 %
mergesort_recursive	1 000 000	63.30 ± 0.11	53.55 ± 0.29	−15.4 %
nbody_single_step	1 024	2.01 ± 0.00	2.02 ± 0.00	+0.6 %
nbody_5_timesteps	1 024	10.04 ± 0.00	10.06 ± 0.03	+0.3 %
parallel_reduction_seq	1 000 000	0.39 ± 0.00	0.38 ± 0.00	−1.9 %
matmul_naive	512	14.23 ± 0.06	18.57 ± 0.02	+30.5 %
fft_forward	65 536	0.93 ± 0.00	1.22 ± 0.00	+30.9 %
fft_roundtrip	65 536	1.93 ± 0.00	2.40 ± 0.00	+24.4 %
parallel_reduction_rayon	1 000 000	0.03 ± 0.00	0.10 ± 0.01	+192.2 %

$^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$; CV % flags omitted for brevity—high-variance entries: `parallel_reduction_rayon` (C++ CV 3.3 %, Rust CV 12.1 %).

Rust shows statistically significant advantages (all $p < 0.001$) on search, hash-table, and sorting benchmarks. The standard-library sort advantage (−80.6 %) likely reflects algorithmic differences between C++ `std::sort` (introsort) and Rust's pattern-defeating quicksort (pdqsort), rather than a language-level performance difference. For matrix multiplication and FFT, C++ retains significant advantages; those benchmarks are sensitive to auto-vectorisation quality and library-level optimisation that may favour the C++ compiler on this microarchitecture. The parallel-reduction Rayon result (+192.2 %, CV 12 %) is the most anomalous: the high CV indicates measurement instability, and the result may reflect suboptimal Rayon configuration rather than an inherent Rust limitation.

4.2 GPU Benchmark Outcomes

Table 2 presents the GPU kernel results with full statistical detail. Timing covers kernel execution only; H2D and D2H transfer times are excluded. Note

that all three GPU benchmarks exhibit high CV % values due to the small trial count ($n = 5$), and results should be interpreted with appropriate caution.

Table 2: GPU kernel performance: C++ vs. Rust (kernel time only, H2D/D2H transfer excluded, $n = 5$ trials per benchmark).

Benchmark	Size	C++ (ms)	Rust (ms)	Overhead
cuda_matmul_tiled	1024	0.0018 ± 0.0009	0.0002 ± 0.0001	−86.5 %
cuda_reduction	1 000 000	0.0633 ± 0.0162	0.0002 ± 0.0001	−99.6 %
cuda_softmax	64×16384	0.0012 ± 0.0012	0.0017 ± 0.0033	+37.7 %

$^{*}p < 0.05$, $^{***}p < 0.001$, ns = not significant. Overhead positive means Rust is slower. CV % = coefficient of variation; values >5 % indicate high measurement instability.

The matrix multiplication and reduction kernels both yield Rust advantages of large to very large effect size ($|d| > 2.49$). However, the C++ matmul CV % of 49 % indicates substantial between-trial variability, which weakens confidence in the point estimates despite the statistically significant p-value. The reduction result is more stable (C++ CV 25.6 %, Rust CV 22.8 %) and aligns with the expected behaviour of a memory-bandwidth-bound kernel where driver overhead—absorbed differently in the C++ and Rust harnesses—may account for part of the difference. The softmax comparison ($p = 0.778$, $d = 0.19$) provides no evidence of a difference between the two implementations; both exhibit very high CV % values (96 % and 194 % respectively), which may reflect warm-up variability in first-call kernel launch latency with only five trials. A larger trial count would be needed before any conclusion could be drawn for the softmax benchmark.

4.3 Safety and Code Metrics

Table 3 reproduces the safety-oriented metadata from the committed `results/raw/sa` `metrics.json` file.

The Rust GPU code is substantially shorter (141 vs. 709 lines), which is partly explained by the hybrid architecture: the Rust GPU runner relies on precompiled CUDA PTX kernels loaded at runtime rather than embedding full kernel source. The zero `unsafe` count indicates that none of the measured source paths—`cpu/rust/` and `gpu/rust/`—contain the `unsafe` keyword. This is consistent with H3, but as noted in Section 3, it does not account for `unsafe` code inside Cargo dependencies or the PTX-compiled kernel code. LOC and `unsafe` counts are only partial proxies for safety and maintainability; they should be treated as supporting evidence rather than conclusive proof that Rust implementations are objectively safer or easier to maintain.

Table 3: Safety and code-size metrics from `safety_metrics.json`.

Metric	Value
C++ CPU LOC	739
C++ GPU LOC	709
Rust CPU LOC	731
Rust GPU LOC	141
Aggregate Rust/C++ LOC ratio	0.60
Recorded Rust `unsafe` blocks (measured paths)	0

LOC = non-blank, non-comment lines in `cpu/` and `gpu/` source trees; `build/` and `target/` excluded. "Measured paths" refers to `cpu/rust/` and `gpu/rust/` only; transitive `unsafe` in Cargo dependencies is not counted.

5 Discussion

5.1 Interpretation of Results

The CPU results are broadly consistent with Ivanov's finding that Rust can match or exceed C++ on optimised standard-library routines [5]. The search, sorting, and hash-table advantages likely reflect stronger algorithmic choices in the Rust standard library (e.g., pdqsort) rather than a pure language overhead difference—a nuance that underscores the importance of distinguishing "language performance" from "standard-library performance." The FFT and naive matrix multiplication disadvantages are consistent with the observation in Martins et al. that Rust's auto-vectorisation can lag behind GCC on certain numerical kernels [8]; this is an active area of improvement in the LLVM back-end shared by both `rustc` and Clang.

The GPU results are more difficult to interpret with confidence. The matmul and reduction outcomes favour Rust by large margins, but both benchmarks show high CV %, and the small trial count ($n = 5$) limits statistical power. These results are directionally consistent with Bychkov and Nikolskiy's observation that Rust GPU drivers can match C++ when kernel code is equivalent [1], but the variability means the magnitude of the advantage is uncertain. The softmax non-result ($p = 0.778$) is the most honest finding in the GPU section: with only five trials and CV values exceeding 96 %, no conclusion about relative performance is warranted.

The zero `unsafe` count across measured Rust paths supports H3 and is consistent with the broader claim in Jung et al. that the Rust type system can enforce safety without runtime cost [6]. However, as Fulton et al. document, safety benefits can be partially offset by increased development effort [3]; LOC alone does not capture that trade-off.

5.2 Implications

For practitioners deciding whether to adopt Rust for a new high-performance system, these results suggest that Rust is a credible choice for workloads similar to search, sorting, and reduction, and that GPU kernel execution performance is not inherently disadvantaged by choosing Rust tooling. The FFT and matrix-multiplication gaps suggest caution for workloads that depend heavily on numerically mature C++ libraries or SIMD optimisations that Rust's LLVM back-end has not yet fully exploited.

5.3 Limitations and Threats to Validity

Several threats limit the generalisability of these results. The benchmark suite is small and selective; results may not extend to workloads such as graph algorithms, sparse linear algebra, or distributed memory communication. All results are tied to one hardware configuration; the Core Ultra 7's hybrid P/E-core topology and the RTX 5060 Ti's Blackwell memory subsystem may produce different relative results than server-class hardware. The GPU trial count ($n = 5$) is insufficient for reliable statistical inference on high-variance kernels; the softmax result cannot be interpreted. Finally, LOC and `unsafe` counts are imperfect proxies. They do not measure compile-time complexity, dependency safety, or the cognitive overhead of the borrow checker.

5.4 Suggestions for Future Research

Future work should increase GPU trial counts to at least 30, add warm-up iterations to reduce first-call latency, and extend the benchmark suite to include sparse and irregular workloads. A profiling study examining auto-vectorisation decisions for the FFT and matmul benchmarks would clarify whether the observed C++ advantage is architectural or compiler-strategy dependent. Measuring compile times (currently `null` in the repository metadata due to toolchain availability constraints) would also strengthen the H3 analysis.

6 Conclusion

6.1 Summary of Main Findings

This study compared Rust and C++ on 15 CPU benchmarks and 3 GPU kernels using a controlled hardware environment, identical input datasets, and rigorous statistical analysis. Rust outperforms C++ on search, sorting, and hash-table workloads by statistically significant margins. C++ retains advantages on FFT, naive matrix multiplication, and one parallel-reduction configuration. GPU kernel comparisons favour Rust for matrix multiplication and

reduction, though high measurement variance constrains confidence in those results, and the softmax benchmark shows no statistically significant difference.

6.2 Research Contribution

This work contributes a unified, reproducible benchmark artifact for Rust-vs-C++ evaluation spanning both CPU and GPU domains—an evaluation that prior literature has not provided in a single, statistically rigorous study. The publicly committed raw data and analysis scripts allow independent verification and extension.

6.3 Closing Thoughts

The evidence from this controlled study supports the view that Rust is a viable language for performance-sensitive CPU and GPU programming. It is not universally faster than C++, and workload characteristics matter substantially. However, the results demonstrate that competitive performance can be achieved without resorting to `unsafe` code in the measured implementations, which suggests that Rust's safety model and high performance are not mutually exclusive goals—at least for the class of workloads examined here.

References

[1] A. Bychkov and V. Nikolskiy. "Rust Language for GPU Programming". In: *Supercomputing – RuSCDays 2022*. Vol. 13708. Lecture Notes in Computer Science. Springer, 2022, pp. 522–532. DOI: `10.1007/978-3-031-22941-1_38`.

[2] Jack Dongarra et al. "The International Exascale Software Project Roadmap". In: *International Journal of High Performance Computing Applications* 25.1 (2011), pp. 3–60. DOI: `10.1177/1094342010391989`.

[3] Kelsey R. Fulton et al. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Seventeenth Symposium on Usable Privacy and Security (SOUPS 2021)*. USENIX Association, 2021, pp. 597–616. URL: `https://www.usenix.org/conference/soups2021/presentation/fulton`.

[4] Andy Georges, Dries Buytaert, and Lieven Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 2007, pp. 57–76. DOI: `10.1145/1297027.1297033`.

[5] N. Ivanov. "Is Rust C++-fast? Benchmarking System Languages on Everyday Routines". In: *arXiv preprint arXiv:2209.09127* (2022). URL: https://arxiv.org/abs/2209.09127.

[6] Ralf Jung et al. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages (POPL)*. Vol. 2. ACM, 2018, 66:1–66:34. DOI: 10.1145/3158154.

[7] B. Köpcke, S. Gorlatch, and M. Steuwer. "Descend: A Safe GPU Systems Programming Language". In: *arXiv preprint arXiv:2305.03448* (2023). URL: https://arxiv.org/abs/2305.03448.

[8] E. M. Martins et al. "NPB-Rust: NAS Parallel Benchmarks in Rust". In: *arXiv preprint arXiv:2502.15536* (2025). URL: https://arxiv.org/abs/2502.15536.

[9] Stack Overflow. *Stack Overflow Developer Survey 2024*. https://survey.stackoverflow.co/2024/. Accessed April 2026. 2024.

[10] Bjarne Stroustrup. "Evolving a Language in and for the Real World: C++ 1991–2006". In: *ACM SIGPLAN Notices* 42.3 (2007), pp. 1–59. DOI: 10.1145/1238844.1238848.

[11] W. Veytsman et al. "Rewrite it in Rust: A Computational Physics Case Study". In: *arXiv preprint arXiv:2410.19146* (2024). URL: https://arxiv.org/abs/2410.19146.

[12] Y. Zhang et al. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Rochester, MI, USA: ACM, 2022. DOI: 10.1145/3551349.3559494.